Lab 7 - Network Simulator 3 Advanced

Motivation: (Network Analysis Techniques)

- **Analytical Approaches**
  - Mathematical analysis / modeling of systems
- **Simulations**
  - Model the system at abstract level via software
  - Various network simulators exist (eg. NS3, OpenNet, QualNet)
- **Emulations (eg. Run Windows on MAC)**
  - HW component that behave like real system (eg. Mininet)
- **Measurements**
  - Active (eg. ping, traceroute) or passive (eg. wireshark)
- **Experimentations**
  - Experiment on a testbed for realism (eg. GENI, PlanetLab, OrbitLab, NAMS Lab)

Motivation for Network Simulations

- **Goals**
  - Build software simulation model of networking systems
  - To analyze/study/improve/develop network archs & protocols
- **Reasons**
  - Real systems are expensive, complex, unavailable
- **Advantages**
  - Relatively easy to set up, deploy and instrument
  - Reproducibility, scalability
- **Disadvantages**
  - Simplified view of complex interactions
  - Could be immensely misleading

How to Run Programs

- ./ns3 run scratch/sample-simulator
- ./ns3 run ‘scratch/sample-simulator --RngRun=2’
  - By default, ns-3 simulations use a fixed seed and run number
  - ns3 is the executable

Network Elements

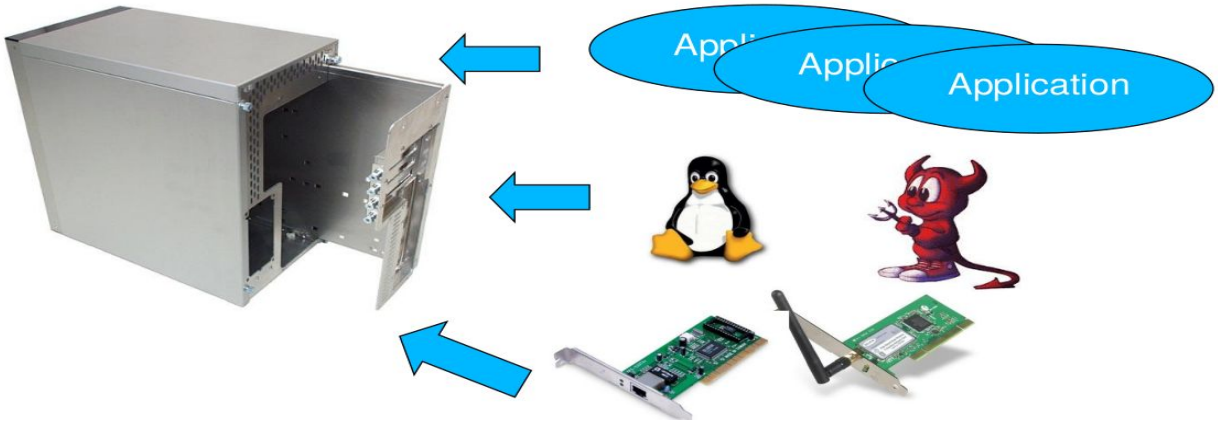
- Basic Components:
- Nodes
- Net Devices (Ethernet NIC)
- Channels
- Application
- Protocol Stack
- Nodes have “Network Devices”
    - Network Devices transfer packets over channels
    - It incorporates L-1 (PHY) & L-2 (MAC)
- The Traffic Control layer in ns-3 is an additional module that intercepts and processes packets between the NetDevices (L2) and any network protocol, such as IP. It's responsible for processing, scheduling, dropping, marking, and policing packets
- Devices interface with L-3 (Network)
- L-3 supports L-4 (Transport)
- L-4 is used by L-5

Abstractions

- Node
- Application
- Channel
- NetDevice
- Packet
- Topology Helpers – aggregate functionality of modules to make common operations easier than using the low-level API
    - Consists of:
        - container objects
        - helper classes

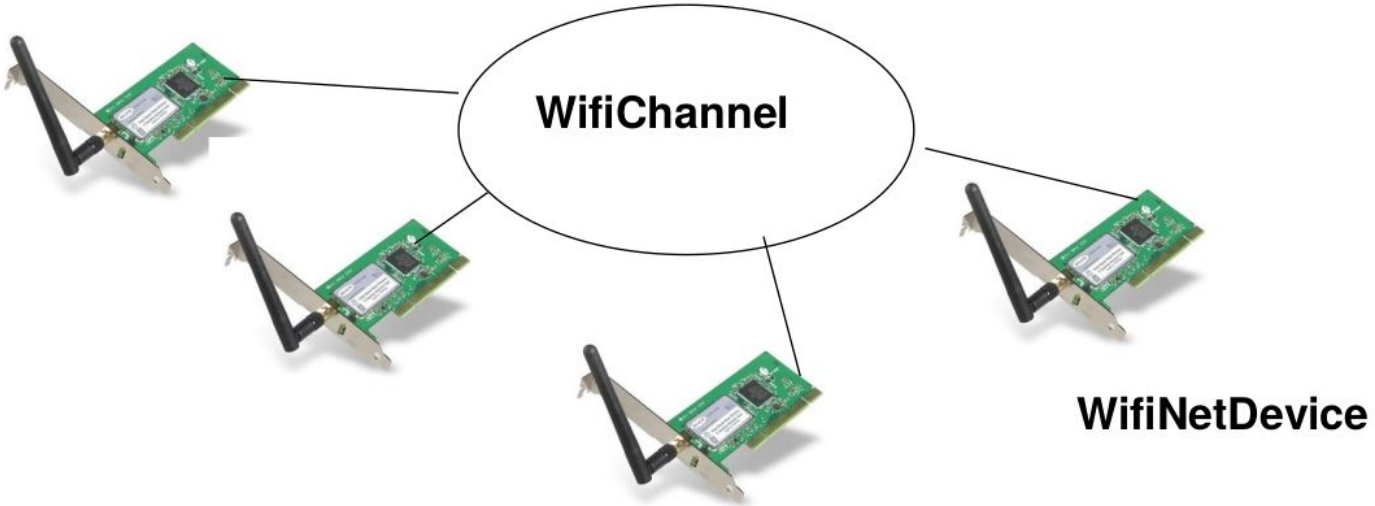
Fundamentals

- Key objects in the simulator are Nodes, Packets, and Channels
- Node is a husk of a computer to which applications, stacks, and NICs (NetDevs) are added
- Nodes are architected for multiple interfaces

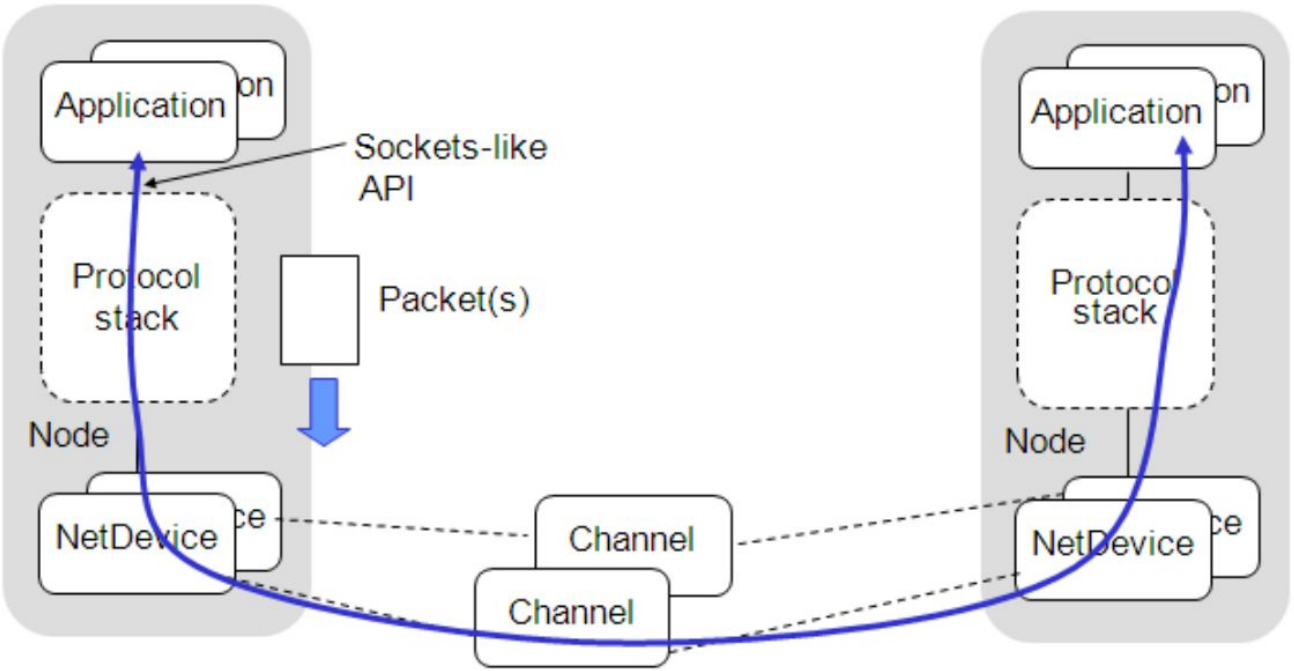


NetDevices and Channels

NetDevices (NICs) are strongly bound to channels of a matching type



Basic Model:


Application
Sockets-like
API
Protocol
stack
Packet(s)
Node
NetDevice
Channel
Channel
Application
Protocol
stack
Node
NetDevice

Structure of NS-3 Program

```
int main (int argc, char *argv[])
{

  // Set default attribute values

  // Parse command-line arguments

  // Configure the topology; nodes, channels, devices, mobility

  // Add (Internet) stack to nodes

  // Configure IP addressing and routing

  // Add and configure applications

  // Configure tracing

  // Run simulation
}
```

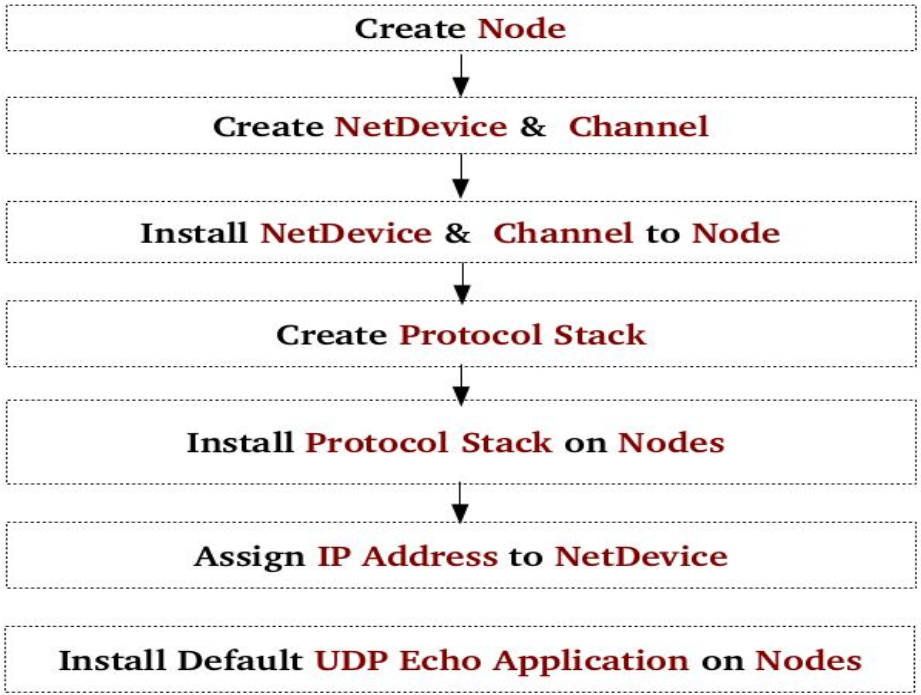
Network Simulation Example:



- Two nodes, one network interface device per node
- Point-to-point link
  - propagation delay: 2ms, data rate:5 Mbps
- Application

- Udp echo client on node 0, Udp echo server on node 1 on port 9
- Payload size of 1024-byte packet
- time interval between packet is 1 s

Flow Chart for writing a program in NS-3



Example Script - 1

```
/* -*- Mode:C++; c-file-style:"gnu"; indent-tabs-mode:nil; -*- */
// GPLv2 Licence …

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");

int main (int argc, char *argv[])
{
  LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
  LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);

  NodeContainer nodes;
  nodes.Create (2);

  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices;
  devices = pointToPoint.Install (nodes);
```

include modules that will be used

ns-3 project namespace

enable and disable console message logging by reference to the name

Create Node

Topology Configuration

Example Script - 2

Install Protocol Stack

Set up Server

Set up Client

```
  InternetStackHelper stack;
  stack.Install (nodes);

  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");

  Ipv4InterfaceContainer interfaces = address.Assign (devices);

  UdpEchoServerHelper echoServer (9);

  ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));
  serverApps.Start (Seconds (1.0));
  serverApps.Stop (Seconds (10.0));

  UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);
  echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
  echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
  echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

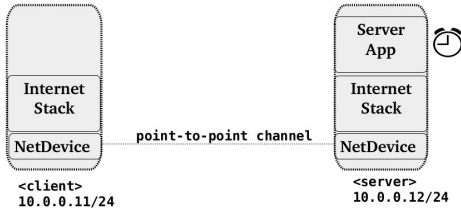
  ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));
  clientApps.Start (Seconds (2.0));
  clientApps.Stop (Seconds (10.0));

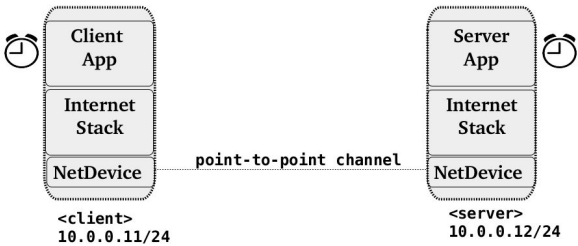
  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```

Set up internet stack



Set up applications





Run the simulation

Running Example

Run the commands given below to execute first.cc

- cp examples/tutorial/first.cc scratch/myfirst.cc
- ./ns3 run “scratch/myfirst.cc”
- Output:

```
Consolidate compiler generated dependencies of target scratch_myfirst
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

Cmd Args

- Add the lines below:
  - int nPackets=0;
  - cmd.AddValue("nPackets", "Number of packets to echo", nPackets);
- ./ns3 run "scratch/myfirst.cc --nPackets=2"
  - This is how one can pass value through CLI to the program.
- ./ns3 run scratch/myfirst.cc --command-template="%s --nPackets=2"
  - Does essentially the same thing in a different way.
- https://www.nsnam.org/docs/release/3.9/tutorial/tutorial_22.html

Playing with myfirst.cc

Execute: ./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::PointToPointNetDevice"

```
Attributes for TypeId ns3::PointToPointNetDevice
    --ns3::PointToPointNetDevice::Address=[ff:ff:ff:ff:ff:ff]
        The MAC address of this device.
    --ns3::PointToPointNetDevice::DataRate=[32768bps]
        The default data rate for point to point links
    --ns3::PointToPointNetDevice::InterframeGap=[+0ns]
        The time to wait between packet (frame) transmissions
    --ns3::PointToPointNetDevice::Mtu=[1500]
        The MAC-level Maximum Transmission Unit
    --ns3::PointToPointNetDevice::ReceiveErrorModel=[0]
        The receiver error model used to simulate packet loss
    --ns3::PointToPointNetDevice::TxQueue=[0]
        A queue to use as the transmit queue in the device.
```

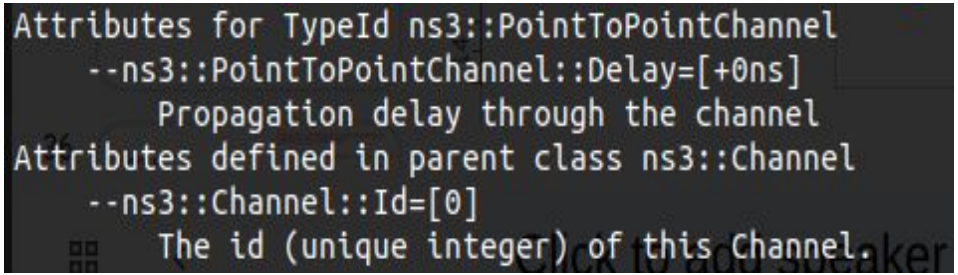
Playing with myfirst.cc...

Execute: ./ns3 run "scrExecute:

./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::PointToPointChannel"

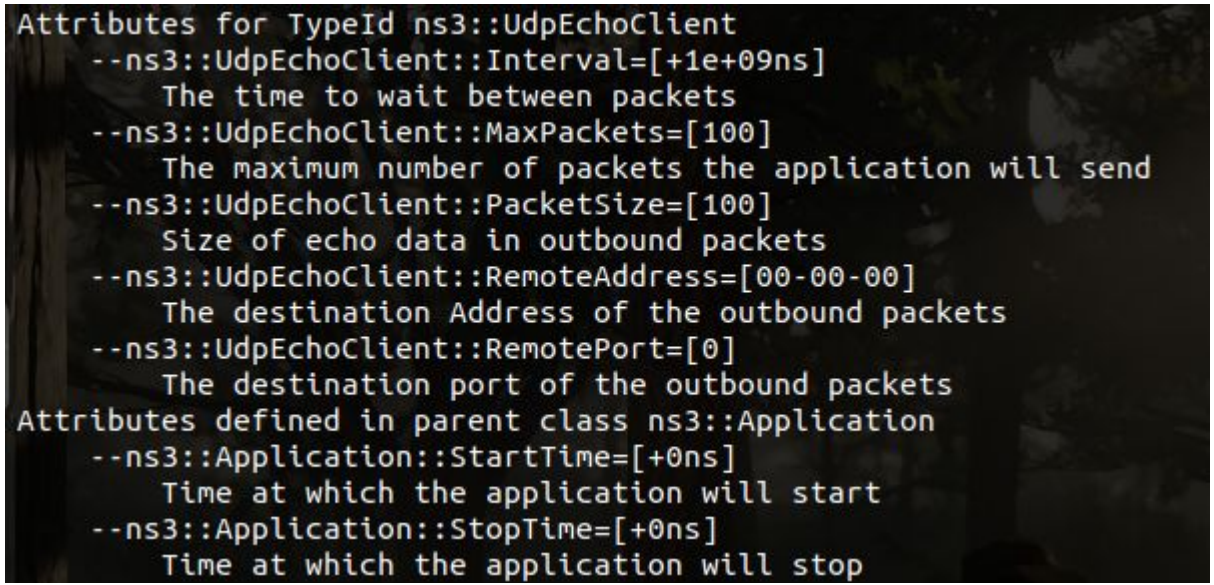
./ns3 run "scratch/myfirst.cc --ns3::PointToPointChannel::Delay=[+0.0ns]"

- Transmission delay through the channel
- Is one of the attributes for PointToPointChannel



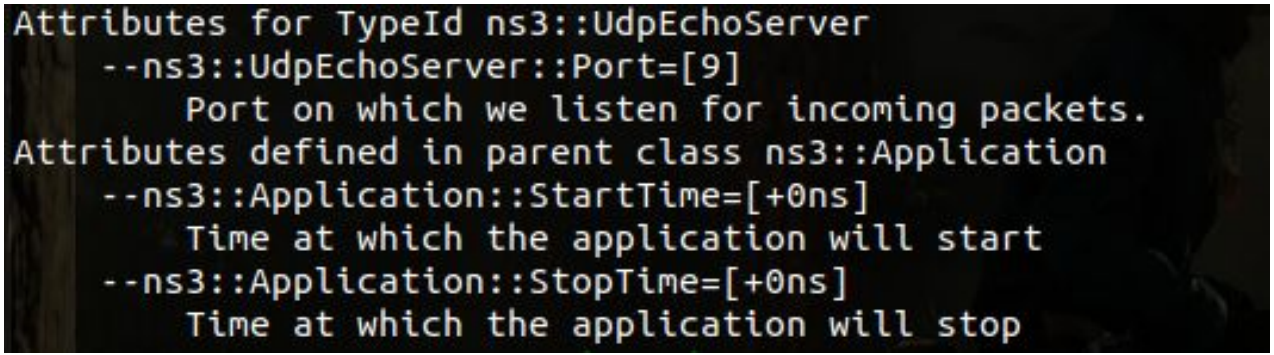
Playing with myfirst.cc…

Execute: ./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::UdpEchoClient"



Playing with myfirst.cc...

Execute: ./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::UdpEchoServer"



Playing with myfirst.cc…

Execute: ./ns3 run "scratch/myfirst.cc --nPackets=2 --ns3::UdpEchoClient::PacketSize=5000"

Output:

```
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

** Note that if the programmer sets any new value to PacketSize in the program, that overrides the above command line arg value of 5000 Bytes. Same thing even with the nPackets user defined variable.

Command Line Argument Override

```
CommandLine cmd (__FILE__);
int nPackets=0;
std::cout<<nPackets<<std::endl;
cmd.AddValue("nPackets", "Number of packets to echo", nPackets);
cmd.Parse (argc, argv);
std::cout<<nPackets<<std::endl;
nPackets=7;
std::cout<<nPackets<<std::endl;
```

Output:

```
0
4
7
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

Logs

- Logs are generally used to get useful information.
- NS-3 provides different levels of logs which are as follows:-

– LOG_INFO

– LOG_FUNCTION

– LOG_LOGIC

– LOG_ALL

– etc.

Enable Logging

- To enable logging :
  - LogComponentEnable(module_name,log_level)
    - Module is any application, UdpEchoClientApplication for instance
  - LogComponentEnableAll(log_level);
    - This enables logging for all the modules
- Example:
  - Module_name
    - ns3::UdpEchoClientApplication
    - ns3::UdpEchoServerApplication
    - And,more
  - Log_level (log level is enabled for the given level and all lower levels)
    - LOG_LEVEL_ERROR
    - LOG_LEVEL_WARN
    - LOG_LEVEL_DEBUG
    - LOG_LEVEL_INFO
    - LOG_LEVEL_FUNCTION
    - LOG_LEVEL_LOGIC
    - LOG_LEVEL_ALL

Getting Help

./ns3 run "sample-simulator.cc --PrintHelp"

```
sample-simulator [General Arguments]

General Arguments:
    --PrintGlobals:              Print the list of globals.
    --PrintGroups:               Print the list of groups.
    --PrintGroup=[group]:        Print all TypeIds of group.
    --PrintTypeIds:              Print all TypeIds.
    --PrintAttributes=[typeid]:  Print all attributes of typeid.
    --PrintVersion:              Print the ns-3 version.
    --PrintHelp:                 Print this help message.
```

Our First Code

- **core-module:** Basic ns-3 core classes such as Simulator, Time, Log, etc.
- **network-module:** Provides classes for network components like nodes, devices, and channels.
- **internet-module:** Adds the internet stack to nodes (TCP/IP protocols).
- point-to-point-module: Facilitates the creation of point-to-point links between nodes.
- **applications-module:** Offers application-layer protocols and helper classes (e.g., UDP Echo Client/Server).
- **netanim-module:** Supports animation and visualization of the network simulation using NetAnim.

Our First Code

- The CommandLine object **cmd** is used to parse command-line arguments.
- The **AddValue** method binds the **tracing** variable to a command-line argument --tracing, allowing users to enable or disable tracing.
- **Parse** processes the actual command-line arguments.
- **NodeContainer nodes:** Creates a container to hold nodes.
- **NodeContainer** is a helper class in ns-3 that manages a collection of Node objects.
- **nodes.Create (2):** Creates two nodes, n0 and n1.

Our First Code

- **PointToPointHelper pointToPoint:** Helper class to configure and create a point-to-point link.
- **SetDeviceAttribute ("DataRate", StringValue ("5Mbps")):** Sets the data rate of the point-to-point link to 5 Mbps.
- **SetChannelAttribute ("Delay", StringValue ("2ms")):** Sets the propagation delay of the link to 2 ms.
- **NetDeviceContainer devices:** A container to hold the network devices.
- **devices = pointToPoint.Install (nodes):** Installs point-to-point devices and connects them to the nodes, creating a point-to-point link between n0 and n1.

Our First Code

- **InternetStackHelper stack:** Helper class to install the internet protocol stack (TCP/IP) on nodes.
- **stack.Install (nodes):** Installs the internet stack on both nodes (n0 and n1).
- **Ipv4AddressHelper address:** Helper class to manage IPv4 addresses.
- **address.SetBase ("10.1.1.0", "255.255.255.0"):** Sets the base network address and subnet mask.
- **Ipv4InterfaceContainer interfaces = address.Assign (devices):** Assigns IP addresses to the devices connected by the point-to-point link.

Our First Code

- **UdpEchoServerHelper echoServer (9):** Creates a UDP echo server application bound to port 9.
- **serverApps = echoServer.Install (nodes.Get (1)):** Installs the server application on node n1.
- **serverApps.Start (Seconds (1.0)):** Starts the server application at 1 second.
- **serverApps.Stop (Seconds (10.0)):** Stops the server application at 10 seconds.
- **UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9):** Creates a UDP echo client application to communicate with the server at IP 10.1.1.2 (address of n1) and port 9.
- **echoClient.SetAttribute ("MaxPackets", UintegerValue (1)):** Configures the client to send only one packet.

Our First Code

- **echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0))):** Sets the time interval between packets to 1 second.
- **echoClient.SetAttribute ("PacketSize", UintegerValue (1024)):** Sets the packet size to 1024 bytes.
- **clientApps = echoClient.Install (nodes.Get (0)):** Installs the client application on node n0.
- **clientApps.Start (Seconds (2.0)):** Starts the client application at 2 seconds.
- **clientApps.Stop (Seconds (10.0)):** Stops the client application at 10 seconds.

What is Flow Monitor?

- The Flow Monitor module goal is to provide a flexible system to measure the performance of network protocols.
- The module uses probes, installed in network nodes, to track the packets exchanged by the nodes, and it will measure a number of parameters.
- Packets are divided according to the flow they belong to, where each flow is defined according to the probe's characteristics (e.g., for IP, a flow is defined as the packets with the same **{protocol, source (IP, port), destination (IP, port)}** tuple.
- The statistics are collected for each flow can be exported in XML format. Moreover, the user can access the probes directly to request specific stats about each flow.

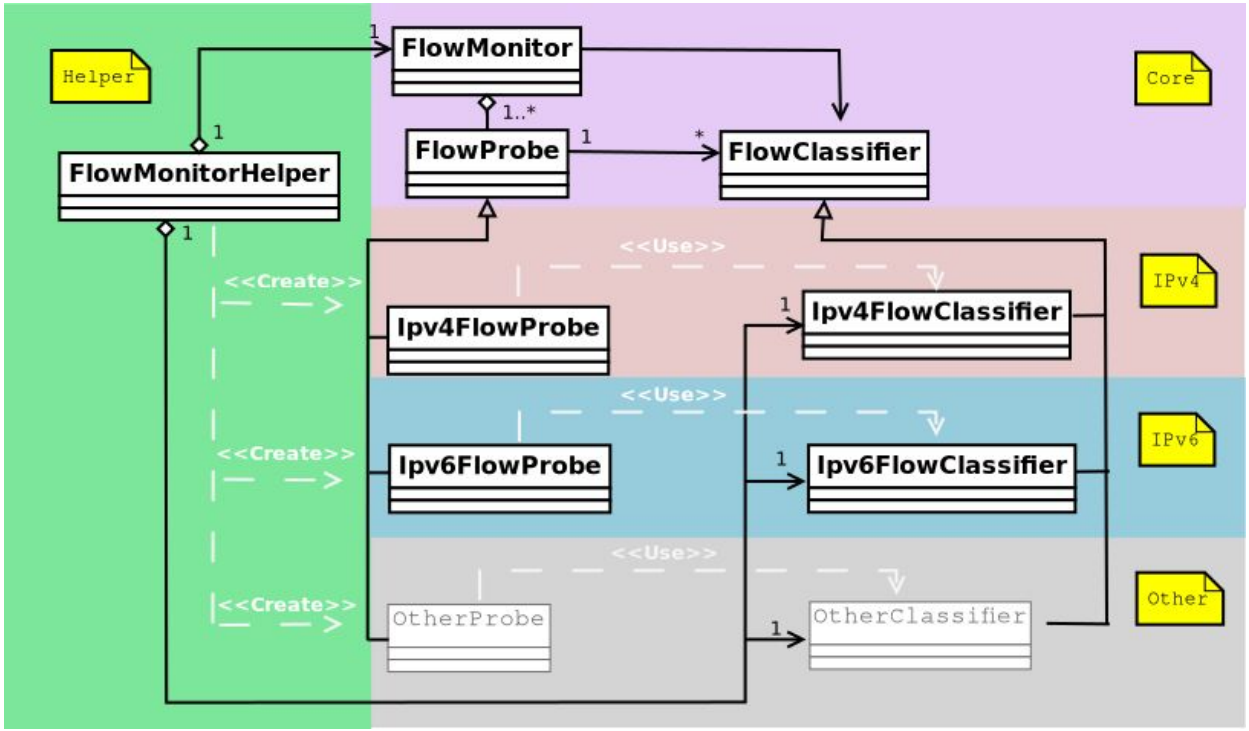
Which Network Parameters Can We Measure?

- At the moment, probes and classifiers are available only for IPv4 and IPv6.
- IPv4 and IPv6 probes will classify packets in four points:
  - When a packet is sent (**SendOutgoing IPv[4,6] traces**)
  - When a packet is forwarded (**UnicastForward IPv[4,6] traces**)
  - When a packet is received (**LocalDeliver IPv[4,6] traces**)
  - When a packet is dropped (**Drop IPv[4,6] traces**)
- Since the packets are tracked at IP level, any retransmission caused by L4 protocols (e.g., TCP) will be seen by the probe as a new packet.
- A Tag will be added to the packet (**ns3::Ipv[4,6]FlowProbeTag**). The tag will carry basic packet's data, useful for the packet's classification.
- Only **L4 (TCP, UDP) packets are, so far, classified**. Moreover, only unicast packets will be classified. These limitations may be removed in the future.

Data Collected For Each Flow:

- **timeFirstTxPacket:** when the first packet in the flow was transmitted;
- **timeLastTxPacket:** when the last packet in the flow was transmitted;
- **timeFirstRxPacket:** when the first packet in the flow was received by an end node;
- **timeLastRxPacket:** when the last packet in the flow was received;
- **delaySum:** the sum of all end-to-end delays for all received packets of the flow;
- **jitterSum:** the sum of all end-to-end delay jitter (delay variation) values for all received packets of the flow, as defined in RFC 3393;
- **txBytes, txPackets:** total number of transmitted bytes / packets for the flow;
- **rxBytes, rxPackets:** total number of received bytes / packets for the flow;
- **lostPackets:** total number of packets that are assumed to be lost (not reported over 10 seconds);
- **timesForwarded:** the number of times a packet has been reportedly forwarded;
- **delayHistogram, jitterHistogram, packetSizeHistogram:** histogram versions for the delay, jitter, and packet sizes, respectively;
- **packetsDropped, bytesDropped:** the number of lost packets and bytes, divided according to the loss reason code (defined in the probe).

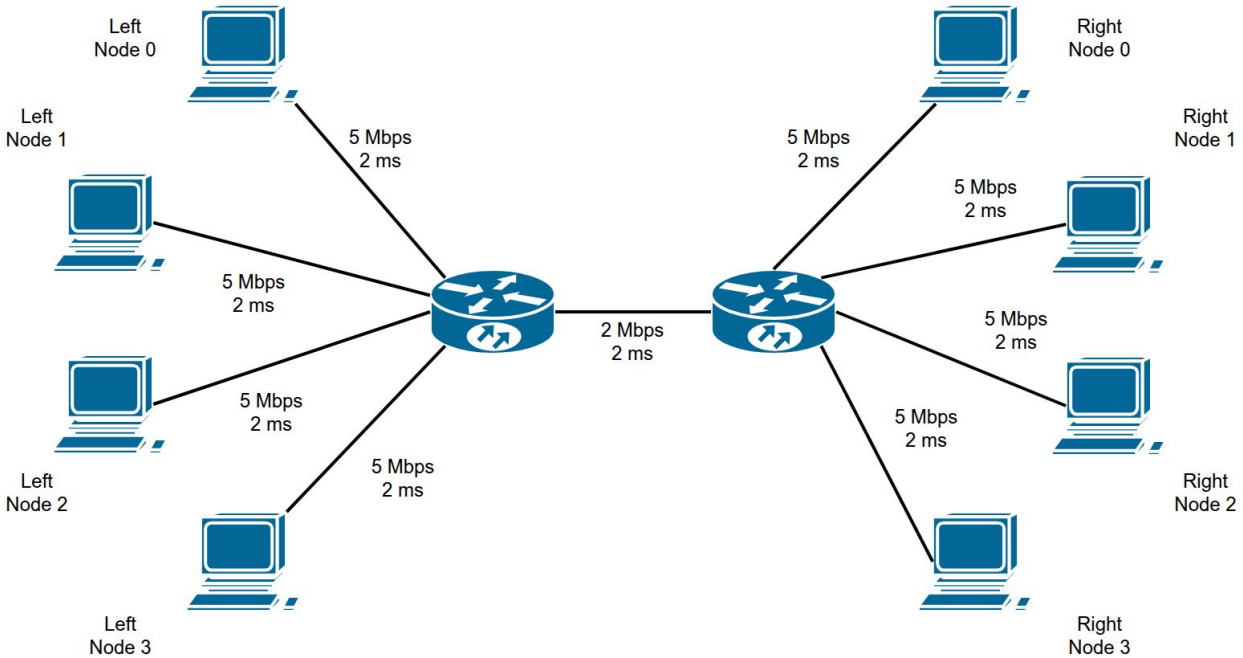
High Level Flow Monitor Architecture



How Flow Monitor Works

- **Setting Up Flow Monitor:** To use Flow Monitor, you need to include the flow-monitor-module in your simulation script and then use the FlowMonitorHelper class to set it up.
- **Attaching Flow Monitor to Nodes:** The FlowMonitorHelper attaches a Flow Monitor instance to all nodes in the simulation, and it automatically tracks all flows that are created.
- **Flow Classification:** The Flow Monitor uses a FlowClassifier (usually an Ipv4FlowClassifier) to classify packets into flows based on their 5-tuple.
- **Statistics Collection:** The Flow Monitor collects statistics from all flows continuously during the simulation. It gathers metrics such as the number of packets sent/received, the number of bytes sent/received, and computes metrics like throughput, delay, and jitter.
- **End of Simulation:** At the end of the simulation, the statistics are typically printed out or saved to a file for analysis.

Demo.cc


Left
Node 0
Left
Node 1
Left
Node 2
Left
Node 3
5 Mbps
2 ms
5 Mbps
2 ms
5 Mbps
2 ms
5 Mbps
2 ms
2 Mbps
2 ms
Right
Node 0
Right
Node 1
Right
Node 2
Right
Node 3
5 Mbps
2 ms
5 Mbps
2 ms
5 Mbps
2 ms
5 Mbps
2 ms

Error Model in NS-3: Reason to Use Error Model

- An ErrorModel in ns-3 simulates packet corruption or loss at the physical/link layer.
- Instead of simulating full physical-layer noise mathematically, ns-3 often uses probabilistic packet errors.
    - Channel → PHY → ErrorModel → NetDevice → Protocol stack
- Location in ns-3:
    - src/network/utils/error-model.h
    - src/network/utils/error-model.cc
- Types of Error Models:
    - RateErrorModel
    - BurstErrorModel
    - ListErrorModel

Error Model in NS-3

- **Adding Error Model:**
  - Ptr<RateErrorModel> em = CreateObject<RateErrorModel>();
  - em->SetAttribute("ErrorRate", DoubleValue(0.05));
  - devices.Get(1)->SetAttribute("ReceiveErrorModel", PointerValue(em));
- **Error Unit Types:**
  - ERROR_UNIT_PACKET
  - ERROR_UNIT_BYTE
  - ERROR_UNIT_BIT
- **Error Unit Type Setting:**
  - em->SetAttribute("ErrorUnit", StringValue("ERROR_UNIT_PACKET"));

Error Model in NS-3

- **RateErrorModel:**
  - Drops packets independently with a fixed probability.
  - Used in introductory labs, abstract wireless links, TCP performance evaluation.
  - Key attributes:
    - **ErrorRate** → probability (0–1)
    - **ErrorUnit** → PACKET / BYTE / BIT
- **Example:**
  - Ptr<RateErrorModel> em = CreateObject<RateErrorModel>();

    em->SetAttribute("ErrorRate", DoubleValue(0.02));

Error Model in NS-3

- **BurstErrorModel:**
  - Produces bursty losses, once an error starts, several consecutive packets are dropped.
  - Used in wireless fading, interference spikes, satellite rain fade
  - Key attributes:
    - **BurstSize** → number of packets in a burst
- **Example:**
  - Ptr<BurstErrorModel> bem = CreateObject<BurstErrorModel>();

    bem->SetAttribute("BurstSize", UintegerValue(5));

Error Model in NS-3

- **ListErrorModel:**
  - Drops packets at predefined packet numbers
  - No randomness
  - Used in debugging, protocol validation, repeatable experiments
- **Example:**
  - Ptr<ListErrorModel> lem = CreateObject<ListErrorModel>();

    std::list<uint32_t> drops = {3, 7, 10};

    lem->SetList(drops);;

Error Model in NS-3

- **Applying Error Model on All the NetDevices Across The Topology:**
  - Ptr<RateErrorModel> em = CreateObject<RateErrorModel>();

    em->SetAttribute("ErrorRate", DoubleValue(0.01));

    Config::Set("/NodeList/*/DeviceList/*/$ns3::PointToPointNetDevice/ReceiveErrorModel", PointerValue(em));
  - Config::SetDefault("ns3::RateErrorModel::ErrorRate", DoubleValue(0.001));

    Config::SetDefault("ns3::RateErrorModel::ErrorUnit", StringValue("ERROR_UNIT_PACKET"));
  - Config::SetDefault("ns3::BurstErrorModel::ErrorRate", DoubleValue(0.01));

    Config::SetDefault("ns3::BurstErrorModel::BurstSize", StringValue("ns3::UniformRandomVariable[Min=1|Max=3]"));

Resources

Web site: http://www.nsnam.org

Mailing list: http://mailman.isi.edu/mailman/listinfo/ns-developers

Tutorial: http://www.nsnam.org/docs/tutorial/tutorial.html

Code server: http://code.nsnam.org

Wiki: http://www.nsnam.org/wiki/index.php/Main_Page

Acknowledgements: Mathieu Lacage, Tom Henderson, www.nsnam.org

Training: https://www.nsnam.org/consortium/activities/training/



THANK YOU